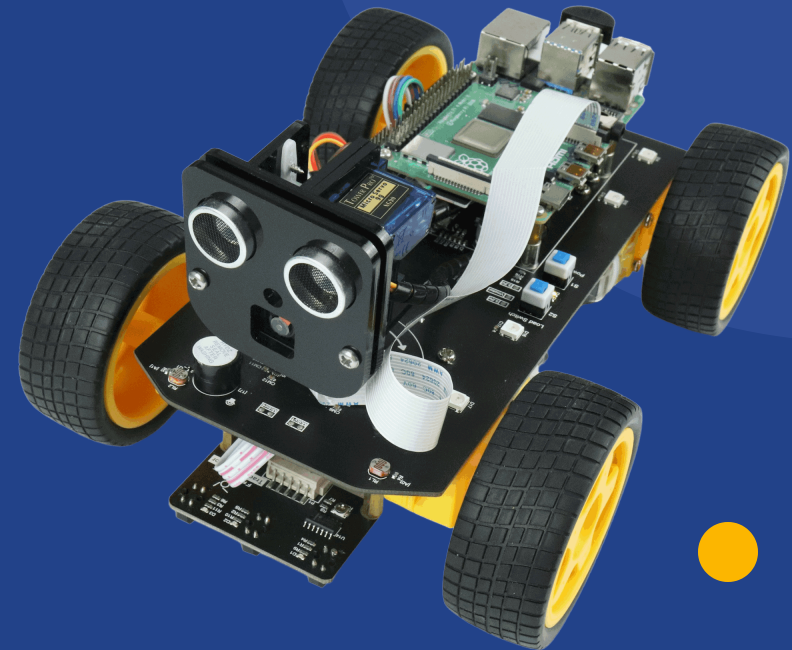




LESSON 1 – COMPONENTS

Machine Learning with Raspberry Pi & Smart Car

Our first code – drive the car's real parts: light the LEDs, turn the head, and measure the distance ahead.



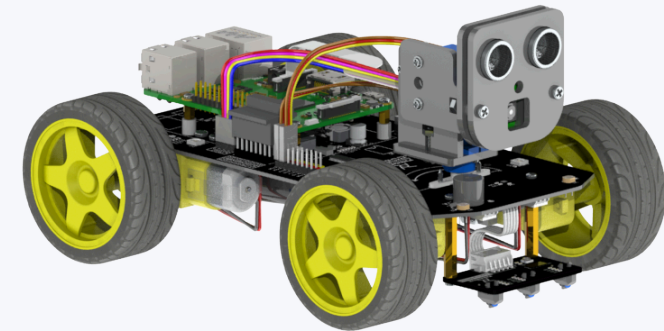


LESSON 1 – COMPONENTS

First, meet the robot's body

A Level 3 course – we write real Python that drives real hardware.

- **Real code, real hardware.** This is Level 3 – Python that controls physical parts, not a simulation.
- **Talk to the parts directly.** Today is the first time we command the car's components from our own code.
- **No machine learning yet.** First we learn the body of the robot; the smart stuff comes later.





LESSON 1 – COMPONENTS

Today's goal: a component tester

One program that runs three mini-demos – built one function at a time.

- 1 **LEDs flash red → green → blue**
- 2 **Servo head sweeps left ↔ right**
- 3 **Ultrasonic sensor prints the distance ahead**



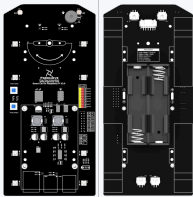
Build it piece by piece, then run the whole thing.



LESSON 1 – COMPONENTS

Three components, one program

You'll drive each of these from Python today.



RGB LEDs

Eight colored lights on the board, controlled as one strip.



Servo (the head)

Turns to an exact angle; it holds the ultrasonic sensor.



Ultrasonic sensor

The "eyes": times a sound echo to measure distance.



LESSON 1 – COMPONENTS

The shape of every lesson script

Learn it once – set up, do the work, clean up – and every future lesson feels familiar.

setup()

Turn the parts on and get them ready.
Runs **once** at the start.



demo functions

The actual work – `show_leds`,
`sweep_head`, `loop`.



destroy()

Leave the car safe – lights off, head centered. **Always** runs.



LESSON 1 – COMPONENTS • THE FILE, CHUNK 1

car_setup: the magic import

The car's hardware code lives in another folder – this line points Python at it.

- **Always first.** `import car_setup` tells Python where the car's modules live.
- After it, short imports like `from led import Led` just work – no long paths.
- You never edit `car_setup.py` – you only import it.

```
lesson_1_components.py

"""Lesson 1: Components.

Run this on the Raspberry Pi (the car).
"""

import car_setup    # tells Python where the modules live
import time          # lets us pause with time.sleep()
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 2

The Pi is case-sensitive

Import names must match the real file names exactly – character for character.

HEADS UP

The file is `servo.py` (lowercase), so `from servo import Servo` works.



`from servo import Servo` – matches the file.



`from Servo import Servo` – works on Windows, **fails on the Pi.**

```
lesson_1_components.py
```

```
# each module controls one piece of hardware
from led import Led
from servo import Servo          # file is "servo.py" (lowercase!)
from ultrasonic import Ultrasonic
```

LESSON 1 – COMPONENTS • THE FILE, CHUNK 3

Settings: a menu switch and the LED bitmask

`DEMO_MODE` picks what runs; the LED bitmask picks which of the 8 LEDs light up – one bit per LED.

```
lesson_1_components.py

# ---- settings you can change ----
DEMO_MODE = "all" # leds / servo / distance / all

# pick which of the 8 LEDs to light up
LED_ALL = 0xFF # all on
LED_OFF = 0x00 # all off
```

Value (hex)	Binary	Which LEDs
0xFF	1111 1111	all 8
0x00	0000 0000	none (off)
0x01	0000 0001	only LED 0 (first)
0x80	1000 0000	only LED 7 (last)
0x0F	0000 1111	first 4
0xAA	1010 1010	every other one

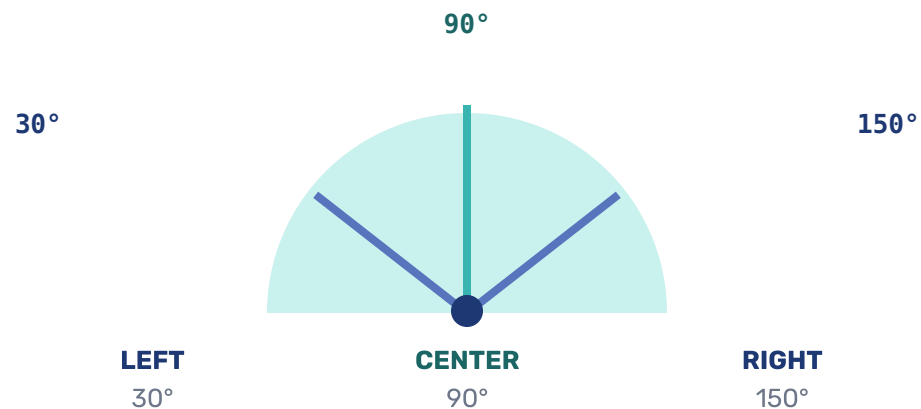
HEX `0x` just means the number is written in **hexadecimal** – each hex digit = **4 bits**, so `0xFF` reads straight off as "8 ones = all LEDs." (`0xFF` = 255, `0x0F` = 15, `0x80` = 128)



LESSON 1 – COMPONENTS • THE FILE, CHUNK 4

Settings: colours and head angles

Colours are (red, green, blue), 0–255. Head angles are degrees – 90 faces straight ahead.



```
lesson_1_components.py
```

```
# colours are (red, green, blue), each 0–255
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)

# head positions in degrees – 90 = straight ahead
HEAD_LEFT = 30
HEAD_CENTER = 90
HEAD_RIGHT = 150

SERVO_DEMO_SECONDS = 5 # how long the sweep runs
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 5

Name the parts, then setup() builds them

Create the hardware objects once, in one place.

- We name the parts and set them to `None` – created later, together.
- `setup()` fills them in and faces the head **straight ahead**.
- `global` means "use the variables above – don't make new ones."
- Channel is the **string** `"0"`, not the number 0.

```
lesson_1_components.py

# ---- car parts (created in setup) ----
led = None
servo = None
ultrasonic = None

def setup():
    """Create the hardware objects once."""
    global led, servo, ultrasonic
    led = Led()           # the LED strip
    servo = Servo()        # the head servo
    ultrasonic = Ultrasonic() # the distance sensor
    servo.setServoPwm("0", HEAD_CENTER) # face forward
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 6

Demo 1: light the LEDs

Loop through red, green, blue – hold each a second – then turn off.



■ red ■ green ■ blue → off

```
lesson_1_components.py

def show_leds():
    """Red, then green, then blue, then off."""
    print("LED demo: red, green, blue, then off")
    for colour in (RED, GREEN, BLUE):
        red, green, blue = colour
        led.ledIndex(LED_ALL, red, green, blue)
        time.sleep(1)          # hold 1 second
    led.ledIndex(LED_ALL, 0, 0, 0) # all off
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 7

Demo 2: sweep the head

Move left → center → right → center until the timer runs out, then re-center.

- Sweep the positions **over and over**.
- Stop after `SERVO_DEMO_SECONDS`, then face forward.
- Check the clock each move so it **stops on time**.

```
lesson_1_components.py

def sweep_head():
    """Sweep, then re-center."""
    print("Servo demo: sweeping", SERVO_DEMO_SECONDS, "s")
    end_time = time.time() + SERVO_DEMO_SECONDS
    while time.time() < end_time:
        for angle in (HEAD_LEFT, HEAD_CENTER, HEAD_RIGHT, HEAD_CENTER):
            if time.time() >= end_time:
                break
            servo.setServoPwm("0", angle)
            time.sleep(0.6)
        servo.setServoPwm("0", HEAD_CENTER)
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 8

Demo 3: read the distance

Print the distance ahead every half-second until you press Ctrl+C.

- `read_distance()` asks the sensor once and returns **centimetres**.
- `loop()` prints it every 0.5s until **Ctrl+C**.
- `while True:` means "repeat forever" – until we stop it.
- Wave a hand in front and watch the number **drop**.

```
lesson_1_components.py

def read_distance():
    """Return the distance ahead, in cm."""
    return ultrasonic.get_distance()

def loop():
    """Print the distance until Ctrl+C."""
    print("Distance demo: press Ctrl+C to stop")
    while True:
        # repeat forever
        distance = read_distance()
        print("Distance ahead:", distance, "cm")
        time.sleep(0.5)
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 9A

Clean up: destroy()

Always leave the car safe – LEDs off, head centered.

- Leave the car **safe and tidy**: lights off, head centered.
- A part may still be `None` if `setup()` failed – so check before using it.
- Real robots should never be left in a weird state.

```
lesson_1_components.py

def destroy():
    """Leave the car safe when the program ends."""
    if led is not None:
        led.ledIndex(LED_ALL, 0, 0, 0)      # LEDs off
    if servo is not None:
        servo.setServoPwm("0", HEAD_CENTER) # center
```



LESSON 1 – COMPONENTS • THE FILE, CHUNK 9B

The main block ties it together

- `if __name__ == "__main__":` runs only when you start **this file** directly.
- **try** runs the chosen demos.
- **except** catches **Ctrl+C** – no scary error.
- **finally always** runs `destroy()` – even after an error.

```
lesson_1_components.py

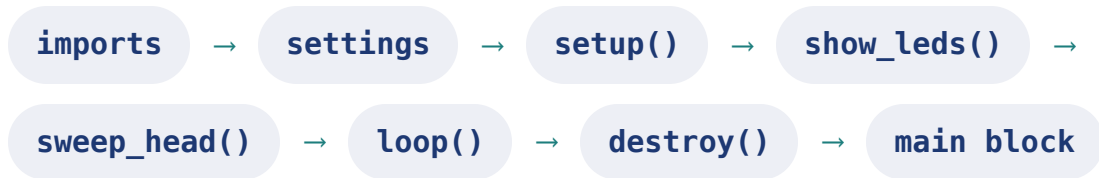
if __name__ == "__main__":
    try:
        setup()
        if DEMO_MODE in ("leds", "all"):
            show_leds()
        if DEMO_MODE in ("servo", "all"):
            sweep_head()
        if DEMO_MODE in ("distance", "all"):
            loop()
        print("Component testing was completed.")
    except KeyboardInterrupt:
        pass # Ctrl+C: shut down quietly
    finally:
        destroy() # always leave the car safe
```



LESSON 1 – COMPONENTS

You built the whole file

Top to bottom, you now have a complete `lesson_1_components.py`.



The same **setup** → **work** → **destroy** shape we'll reuse all course long.



Nine chunks, one working program.



LESSON 1 – COMPONENTS

Where to write & save it

Edit and save in Thonny – then run from the terminal.

- **Write it in Thonny** – the Pi's built-in Python editor (already on Raspberry Pi OS). Open it on the Pi desktop: **Menu** → **Programming** → **Thonny**.
- **Save in** `Smart-Car-Level-3/Code/User` – next to `car_setup.py`, so the imports resolve. Keep the name `lesson_1_components.py`.
- All course long: **edit & save in Thonny** → **run from the terminal**.

**NOT THONNY'S RUN BUTTON**

These lessons need `sudo` (root) for the hardware, so we run them from the **terminal**, not Thonny's green Run.



LESSON 1 – COMPONENTS

Run it on the Pi

Saved in Code/User – now run it from the Pi's terminal (it needs sudo).

```
Pi terminal  
  
cd Smart-Car-Level-3/Code/User  
sudo python3 lesson_1_components.py
```

WHY SUDO?

The LED strip library needs **root** to reach the hardware.

- Expect: LEDs cycle colours → head sweeps → distance prints.
- Press **Ctrl+C** to stop the distance loop.



LESSON 1 – COMPONENTS

Try it yourself

Change a few settings and make the demo your own.

- Set `DEMO_MODE` to `"leds"`, `"servo"`, or `"distance"` to run one part.
- Add your own colour – e.g. purple `(160, 0, 255)`.
- Make the head sweep **wider** or **slower** (`HEAD_LEFT`, `time.sleep`).

CHALLENGE

Red when close

Light the LEDs red when something is closer than **15 cm**. (A taste of the sensor → action logic in later lessons.)



LESSON 1 – COMPONENTS

Common issues

If something's off, it's almost always one of these.



Can't open /dev/mem • mmap() failed

You forgot `sudo` – the LED strip needs root.



No module named 'Servo'

Casing! It's `from servo import Servo` (lowercase file).



Head buzzes or jitters

Low battery, or it's fighting a wall – check power, give it room.



No distance / weird numbers

Sensor wires loose, or it's pointed at a soft/angled surface.



Stuck?

Check casing and `sudo` first – they cover most of today's snags. Then ask.



LESSON 1 – COMPONENTS

You did it!

You drove three components from one Python program.

- ✓ Drove the LEDs, servo head and ultrasonic sensor
- ✓ Learned the setup → work → destroy skeleton
- ✓ Saw why `import car_setup` comes first

UP NEXT

Lesson 2 Make it move

Spin the wheels, and see how the car's commands travel from a client to the server.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

- 1 Post the after-class recap**
Summarise what you covered on the class page so students can catch up.
- 2 Log absent students**
Post anyone who missed class in the **#attendance** Discord channel.
- 3 Set up the next class**
Charge the cars and share any prep before Lesson 2 (it actually drives!).

EVERY CLASS

- ✓ Post a class announcement
- ✓ Introductions if needed
- ✓ Note absences in #attendance